Universidad del Valle
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación




# Guía:
## Herencia




El siguiente documento ha sido elaborado tomando fragmentos de documentos previamente publicados, así como imágenes y ejercicios, sobre los cuales se han realizado traducciones libres y adaptaciones con el propósito de desarrollar el material para el curso de Fundamentos de Programación Orientada a Objetos de la Universidad del Valle. Se ha respetado la autoría original de los materiales, y se han proporcionado las referencias correspondientes dentro del documento. El uso de este material se restringe estrictamente al marco académico y formativo de los estudiantes que cursen la asignatura, y no tiene fines comerciales. Queda prohibida su copia o distribución fuera del contexto de dicho curso. *Una primera versión de esta guía se realizó con el acompañamiento de los estudiantes de doctorado: Giovanny Hidalgo y José Llanos.*

# Herencia

En programación orientada a objetos, la herencia es un mecanismo permite definir una clase base que encapsula características y comportamientos comunes a un conjunto de clases relacionadas [1]. Las clases derivadas pueden entonces heredar estas propiedades y métodos, extendiéndose con características específicas que sean exclusivas de cada subclase [2]. La herencia permite la reutilización del código, la modularización y especialización de las clases [3].

La herencia en programación orientada a objetos puede entenderse fácilmente si la comparamos con la genética. Por ejemplo, cuando una persona recibe de sus padres ciertos rasgos genéticos, como el color del cabello, se dice que ha heredado esa característica. Esto también se aplica a actitudes o comportamientos que se transmiten de una generación a otra. Por ejemplo, cuando se dicen de forma coloquial: tiene el mismo genio del abuelo. De manera análoga, en programación, una clase puede heredar atributos y comportamientos de otra, permitiendo el aprovechamiento y la reutilización de código existente [4].

Así, una clase actúa como un modelo que define las propiedades y comportamientos de una entidad específica. Cuando una clase hereda de otra, adquiere todas las características y comportamientos de la clase base, con la posibilidad de extender su funcionalidad mediante la incorporación de nuevos atributos o métodos, e incluso modificar aquellos heredados para adaptarlos a sus necesidades particulares [1].

Entonces la herencia es un mecanismo que permite crear nuevas clases, denominadas clases derivadas o subclases, a partir de clases existentes, llamadas clases base o superclases. Este concepto es fundamental para promover la reutilización de código, ya que las subclases heredan los atributos y métodos de la superclase, lo que les permite aprovechar comportamientos ya definidos sin necesidad de reescribirlos [4]. Además, las subclases pueden

extender o modificar esta funcionalidad mediante la sobreescritura (override) de métodos o la adición de nuevas características [5]. La herencia facilita la organización jerárquica de las clases y soporta el principio de sustitución de Liskov, permitiendo que los objetos de una subclase sean utilizados en lugar de objetos de su superclase, lo cual resulta en sistemas más flexibles y mantenibles [6].

Supongamos que estamos desarrollando un videojuego en el que los jugadores pueden ser de diferentes tipos, como guerreros o magos. Podemos definir una clase base llamada Jugador, que contiene atributos y métodos comunes, como nombre, nivel y el método atacar(). A partir de esta clase, podemos crear subclases como Guerrero y Mago, que heredan de Jugador y agregan características únicas.

## Relación de herencia

En un diagrama de clases, la relación de herencia se representa mediante una flecha dirigida hacia la clase base o superclase, con un triángulo en su extremo [2]. Esta flecha indica que una clase derivada hereda los atributos y métodos de la clase base, lo que significa que la clase hija puede acceder a la funcionalidad definida en la clase base, además de agregar o modificar características propias. La herencia permite que las clases compartan comportamientos comunes, lo que facilita la reutilización de código y la creación de jerarquías más especializadas y específicas dentro del sistema. En la Figura 1. presenta un ejemplo de las relaciones de herencia utilizadas para crear un tablero que contiene fichas de tres tipos: abismo, camino y salida. De igual manera, está la relación de herencia entre personaje y avatar, ver repositorio [7].

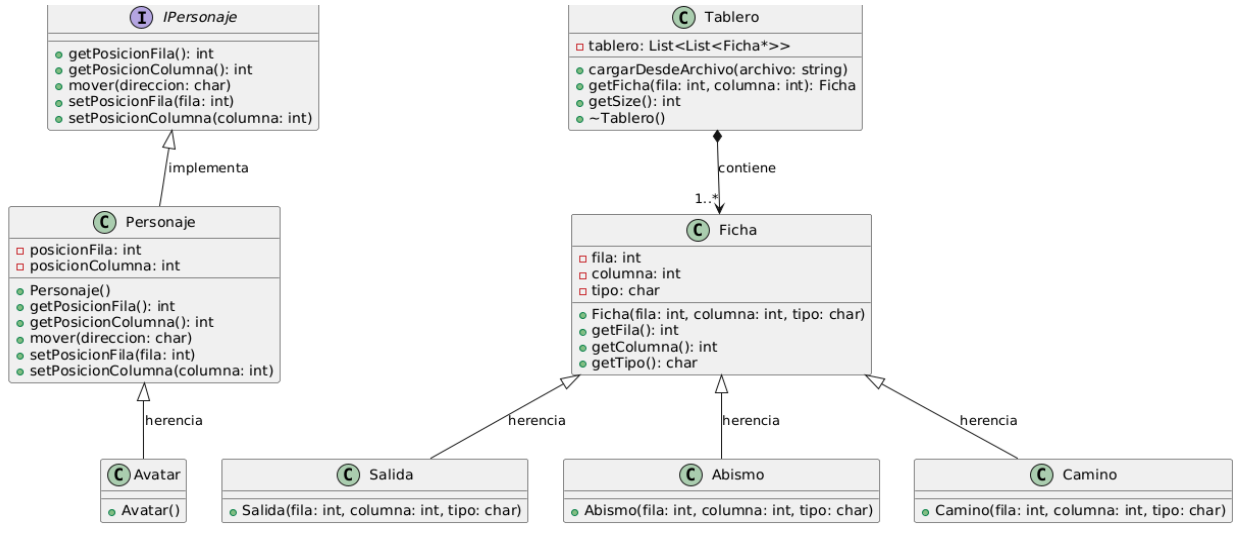



*Figura 1. Herencia: tablero que contiene fichas de tres tipos: abismo, camino y salida*




El diagrama de clases muestra un diseño que utiliza herencia, ver Figura 1. La herencia de clases como Ficha y Personaje permite la especialización sin duplicar código. Este enfoque permite construir un sistema extensible y mantenible, lo cual permite implementar un juego modular basado en tableros y personajes. Así, el diagrama presenta un sistema de clases para un juego basado en un tablero, con un enfoque en la organización orientada a objetos. A continuación, se describen las relaciones de herencia:

Herencia en la clase `Ficha`

- Clases derivadas:
  - `Abismo`
  - `Camino`
  - `Salida`
- Relación: Estas clases heredan de `Ficha`, lo que significa que comparten los atributos (`fila`, `columna`, `tipo`) y los métodos de la clase base. Cada subclase puede especializarse para representar diferentes tipos de fichas en el tablero:
  - `Abismo`: Representa un obstáculo o peligro.
  - `Camino`: Representa una celda transitable.
  - `Salida`: Representa el objetivo del juego.

Herencia en la clase `Personaje`

- Clase derivada:
  - `Avatar`
- Relación: `Avatar` hereda de `Personaje`, indicando que es una implementación específica del concepto general de personaje. Comparte atributos como `posicionFila` y `posicionColumna`, además de los métodos para moverse y establecer su posición.

Adicionalmente en el diagrama de clases presentado en la Figura 1. está presente el concepto de interfaz. Una interfaz es una abstracción que define un conjunto de métodos que una clase debe implementar, sin proporcionar detalles de su implementación [8]. En programación orientada a objetos, las interfaces permiten establecer contratos que las clases deben cumplir, garantizando consistencia en el comportamiento de diferentes objetos, que posteriormente las implementen. Aunque las interfaces no contienen lógica ni atributos, facilitan el diseño modular y la reutilización de código [4]. Su relación con la herencia radica en que una clase que implementa una interfaz hereda indirectamente su comportamiento, asegurando que los métodos definidos en la interfaz sean implementados, permitiendo que sean utilizadas de manera uniforme en un sistema [5], de esta forma en el diagrama de la Figura 1. encontramos la interfaz IPersonaje:




- `Personaje` implementa `IPersonaje`:
    - La clase `Personaje` proporciona la implementación concreta de los métodos definidos en la interfaz `IPersonaje`. Esto asegura que los métodos esenciales, como `mover` y `getPosicionFila`, están disponibles en cualquier clase que implemente esta interfaz.
    - La clase `Personaje` cumple con el contrato de la interfaz `IPersonaje`, lo que garantiza la funcionalidad de moverse en el tablero y obtener información del personaje.

Especialización y diseño basado en interfaces: el uso de interfaces como `IPersonaje` permite flexibilidad en el diseño. Esto posibilita reemplazar implementaciones concretas por otras nuevas sin afectar al resto del sistema. Por ejemplo si queremos crear un nuevo personaje que se mueve en diagonal o un personaje que sea controlado por la CPU, se puede adicionar la clase `PersonajeAtajo` o `PersonajeCPU` que implemente de `IPersonaje`. Esto permite no modificar el código ya existente y añadir las nuevas funcionalidades. Al hacer modificaciones al programa, es posible controlar la consistencia del diseño, pues no es necesario tocar el código preexistente, y la interfaz garantiza que las nuevas clases sean coherentes con el diseño previo.

## *Relación de herencia: es parecido a*

Si la clase derivada añade nuevas funciones o atributos, es una relación "es parecido a" [2]. Cuando una clase derivada introduce funcionalidades adicionales, su comportamiento general puede diferir de la clase base. En este caso, aunque la clase derivada se relaciona con la base, no es exactamente la misma.

**Base:** `Vehiculo`
**Derivada:** `Camion` (añade capacidad de carga)
Relación: "El Camion es parecido a un Vehículo, pero especializado."

## *Relación de herencia : es un*

Si la clase derivada no añade nuevas funciones ni atributos, es una relación "es un" [2]. Si la clase derivada simplemente hereda todos los atributos y métodos sin modificaciones ni extensiones, la relación es más estricta, y la clase derivada puede considerarse como un subtipo directo de la clase base.

*Base: `IVehiculo`*
*Derivada: `Vehiculo` (implementa lo que define la interfaz)*
*Relación: "El vehículo es una abstracción de la interfaz vehículo."*




## *Uso de la Relación de Herencia*

En la **programación orientada a objetos (POO)**, establecer relaciones entre clases es esencial para estructurar y organizar el código de manera eficiente, favoreciendo la reutilización, la extensibilidad y el mantenimiento [1], [2]. Estas relaciones permiten modelar jerarquías y comportamientos compartidos, maximizando la cohesión del sistema y reduciendo la duplicación de código[1], [2]. La herencia, en particular, es una relación fundamental en POO y se utiliza para definir nuevas clases basadas en las clases existentes.

La herencia permite extender las funcionalidades de una clase base sin necesidad de modificar su código original. Esto es útil cuando la clase base ya está probada o utilizada en otros contextos, ya que evita introducir errores en implementaciones existentes [9]. **Esto facilita la incorporación de nuevas funcionalidades sin comprometer el diseño original, favoreciendo el principio de *abierto/cerrado* (OCP) de SOLID.**

Una subclase deriva de la clase base, adquiriendo automáticamente todos sus atributos y métodos públicos y protegidos. Esto crea una jerarquía que representa relaciones de especialización entre las clases [1], [2]. Lo cual permite definir una estructura jerárquica donde las subclases pueden especializar o extender el comportamiento de la clase base según sea necesario.

Las subclases heredan los atributos definidos en la clase base, siempre que sean públicos o protegidos. Esto asegura que las subclases puedan reutilizar y manipular datos ya definidos **[1], [2]**. De esta manera, se reduce la duplicación de datos en las subclases, asegurando consistencia en la estructura y comportamiento de los objetos.

Las subclases heredan los métodos definidos en la clase base, lo que les permite utilizarlos directamente o sobreescribirlos mediante polimorfismo (si el método es virtual en lenguajes como C++ o está anotado como `override` en otros) **[1], [2]**. **Por lo tanto las subclases pueden especializar comportamientos mientras mantienen una interfaz consistente, favoreciendo la reutilización de código y la personalización.**

La herencia permite reutilizar código existente de la clase base en las subclases, evitando reescribir funcionalidades comunes. Las subclases pueden agregar nuevos comportamientos o modificar los existentes sin duplicar el código base [1], [2]. Así se reduce la redundancia y el tiempo de desarrollo, además de facilitar el mantenimiento al centralizar el código compartido en una clase base.

La herencia modela una relación **es-un**, donde la subclase es una especialización del tipo definido por la clase base, ejemplo `Avatar` **es-un** `Personaje`, este tipo de relación permite que las subclases sean tratadas como instancias del tipo base, habilitando el **polimorfismo** [2],

[9]. Esto permite escribir código más general y flexible que puede trabajar con cualquier subtipo sin preocuparse por sus implementaciones específicas.

## *La implementación de herencia en C++*

En C++ la implementación se realiza declarando una clase derivada que hereda de una o más clases base. La sintaxis permite especificar el tipo de herencia (pública, protegida o privada), lo que determina el nivel de acceso a los miembros de la clase base dentro de la clase derivada y para los usuarios de la clase derivada [2].

La Forma general de declarar clases derivadas es: ***class*** `ClaseDerivada` *:* `modo_acceso` `ClaseBase` *{ // Miembros adicionales de ClaseDerivada };*

**Elementos de la declaración:**

1. ***class:*** palabra reservada C++ para definir una clase.

2. `ClaseDerivada`:
   Es el nombre de la nueva clase que se está definiendo. Esta clase se hereda de la clase base especificada.
3. `modo_acceso`:
   Indica cómo se heredan los miembros de la clase base en la clase derivada. Puede ser uno de los siguientes:
   - `public`: Los miembros públicos de la clase base conservan su nivel de acceso.
   - **Nota**: Si no se especifica el modo de acceso, la herencia es **privada** por defecto.
   - **Nota**: se recomienda utilizar la herencia como **pública.**
4. `ClaseBase`:
   Es el nombre de la clase de la cual se está heredando. Puede ser una sola clase o una lista separada por comas en caso de herencia múltiple.

## *Herencia: definición del control de acceso de la clase public, protected y private.*

En la programación orientada a objetos, la herencia es el mecanismo que permite que una clase derivada hereda atributos y métodos de una clase base. Sin embargo, el nivel de acceso a estos elementos en la clase derivada depende del especificador de acceso definido en la

clase base y el tipo de herencia utilizada (**public**, **protected**, o **private**). Esto permite controlar qué partes de la clase base son visibles y accesibles en la clase derivada [2].

1. **Acceso privado:** Los miembros privados de la clase base no son accesibles directamente en la clase derivada, la herencia no tiene efecto en este caso. Estos miembros sólo pueden ser manipulados a través de funciones públicas o protegidas de la clase base. Por ejemplo, si un atributo se declara como privado en la clase base, la clase derivada no podrá acceder a él directamente.
2. **Acceso protegido:** Los miembros protegidos de la clase base son accesibles en la clase derivada, pero permanecen inaccesibles desde fuera de la jerarquía de herencia. Esto es útil cuando se desea que ciertos atributos o métodos sean compartidos entre la clase base y sus derivadas, pero no expuestos a otras clases.
3. **Acceso público:** Los miembros públicos de la clase base son accesibles directamente en la clase derivada y también para cualquier otra clase que interactúe con la derivada.

## *Herencia múltiple*

En C++ la **herencia múltiple** significa que una clase derivada puede heredar desde más de una clase base. En el siguiente recuadro se presenta un ejemplo de herencia múltiple, ver Recuadro 1.

La forma general de declarar la herencia múltiple es: ***class*** `ClaseDerivada` *:* `modo_acceso1` `ClaseBase1`*,* `modo_acceso2` `ClaseBase2` *{/// Miembros adicionales};*

El ejemplo presentado en el Recuadro 1, muestra las ventajas del uso de la herencia, pues permite modelar objetos que combinan características de diferentes clases base, de igual manera evita la duplicación de código al reutilizar las clases base y facilita la creación de estructuras jerárquicas complejas. En las siguientes líneas se explica el ejemplo:

1. **Clases base:**
   - `Vehiculo`: Proporciona atributos y métodos relacionados con vehículos, como la velocidad máxima.
   - `Electronico`: Define características propias de dispositivos electrónicos, como la marca.
2. **Clase derivada:**
   - `VehiculoElectrico` hereda de ambas clases base. Integra la funcionalidad de un vehículo y un dispositivo electrónico, añadiendo su propia característica: la autonomía.

3. **Herencia múltiple:**
   La clase derivada **`VehiculoElectrico`** combina los atributos y métodos de ambas clases base. Los métodos de las clases base se invocan directamente desde la clase derivada para mostrar sus datos.
4. **Uso del constructor:**
   El constructor de **`VehiculoElectrico`** utiliza la **lista de inicialización** para llamar a los constructores de las clases base y establecer valores iniciales para los atributos heredados.

```cpp
#include <iostream>
using namespace std;

// Clase base 1
class Vehiculo {
protected:
    int velocidadMaxima;

public:
    Vehiculo(int velocidad) : velocidadMaxima(velocidad) {}
    void mostrarVelocidadMaxima() const {
        cout << "Velocidad máxima: " << velocidadMaxima << " km/h" << endl;
    }
};

// Clase base 2
class Electronico {
protected:
    string marca;

public:
    Electronico(string m) : marca(m) {}
```

```
23      void mostrarMarca() const {
24          cout << "Marca: " << marca << endl;
25      }
26  };
27
28  // Clase derivada
29  class VehiculoElectrico : public Vehiculo, public Electronico {
30  private:
31      int autonomia; // Autonomía en kilómetros
32
33  public:
34      VehiculoElectrico(int velocidad, string m, int km)
35          : Vehiculo(velocidad), Electronico(m), autonomia(km) {}
36
37      void mostrarDetalles() const {
38          mostrarVelocidadMaxima(); // Método heredado de Vehiculo
39          mostrarMarca();           // Método heredado de Electronico
40          cout << "Autonomía: " << autonomia << " km" << endl;
41      }
42  };
43
44  int main() {
45      // Crear un objeto de VehiculoElectrico
46      VehiculoElectrico tesla(250, "Tesla", 500);
47      tesla.mostrarDetalles(); // Mostrar información del vehículo
    eléctrico
48
49      return 0;
50  }
```

*Recuadro 1. Herencia múltiple*

## *Lo que se hereda y lo que no se hereda*

Todas las funciones y atributos de la clase base se heredan en la clase derivada, excepto:

1. El operador de asignación (`operator=`): El operador de asignación predeterminado no se hereda. Si es necesario en la clase derivada, debe redefinirse explícitamente para adaptarlo a los nuevos atributos o la lógica de asignación.




2. Los constructores y destructores: los constructores y destructores de la clase base no se heredan automáticamente. En su lugar:
    - El **constructor** de la clase derivada debe **invocar explícitamente** el constructor de la clase base, usando la lista de inicialización.
    - El **destructor** de la clase derivada invoca automáticamente el destructor de la clase base al destruirse.
3. Constructor de copia (`Clase(const Clase &`**:** este tampoco se hereda automáticamente. Si es necesario, la clase derivada debe definir explícitamente su propio constructor de copia, llamando al de la clase base para garantizar una copia completa.

## *Cuando no heredar en programación orientada a objetos*

La herencia en programación orientada a objetos es una herramienta clave para evitar duplicación de código, ya que permite que una clase derivada incorpore atributos y métodos de una clase base. Sin embargo, su uso debe ser moderado y bien justificado. Uno de los principales problemas es la fragilidad de la clase base [9], ya que cualquier cambio en ella puede romper las clases derivadas, generando un fuerte acoplamiento jerárquico entre las clases. Además, se corre el riesgo de heredar funcionalidades innecesarias, como en el "problema del banano-gorila-jungla" identificado por Joe Armstrong: al buscar algo simple (como un plátano), puedes terminar arrastrando complejidades innecesarias (como un gorila y su entorno) [9].

Para evitar estos inconvenientes, la herencia debe utilizarse únicamente cuando se identifica una clara relación de tipo "es un" o “es parecido a” entre las clases. Por ejemplo, Mario-Bross es un Personaje, por lo que la herencia es apropiada; en cambio, una VehiculoElectrico no es un Batería, por lo que no debe usarse. También se recomienda heredar de clases abstractas que definen una interfaz mínima, en lugar de clases concretas con múltiples funcionalidades. Esto no solo reduce el riesgo de heredar características innecesarias, sino que también fomenta un diseño más modular y flexible, evitando el sobre acoplamiento y mejorando la mantenibilidad del código.

## *Herencia: constructores y destructores*

Cuando una clase derivada hereda de una clase base, el constructor de la clase derivada debe llamar explícitamente al constructor de la clase base si este requiere argumentos [2]. Esta llamada explícita se realiza en la lista de inicialización del constructor de la clase derivada. De esta forma, los atributos y recursos de la clase base se inicializan correctamente antes de que se ejecute el código específico de la clase derivada. Por ejemplo, si la clase base tiene un

constructor parametrizado, la clase derivada debe proporcionar los valores necesarios durante su construcción. Esto garantiza que la jerarquía de herencia se respete en el proceso de inicialización.

Cuando la clase base tiene un constructor por defecto (sin parámetros), no es necesario llamar explícitamente a este constructor desde la clase derivada. En estos casos, el compilador inserta implícitamente la llamada al constructor por defecto de la clase base [2]. Esto simplifica el código, ya que no se necesita escribir una lista de inicialización adicional. De manera similar, los destructores se manejan automáticamente. Cuando el destructor de la clase derivada se ejecuta, llama implícitamente al destructor de la clase base para liberar correctamente cualquier recurso asociado con la base. Este comportamiento es automático y no requiere ninguna intervención por parte del programador.

El orden de ejecución de los constructores y destructores sigue un flujo lógico: los constructores se llaman de la clase base hacia la clase derivada, mientras que los destructores se llaman en el orden inverso, de la clase derivada hacia la clase base. El orden de llamada a constructores y destructores es el lógico:

1. Constructor clase Base
2. Constructor clase Derivada
3. ...
4. Destructor clase Derivada
5. Destructor clase Base

## Referencias


[1] J. A. Villalobos S., *Fundamentos de programación: aprendizaje activo basado en casos : un enfoque moderno usando Java, UML, objetos y eclipse*. Colombia: Pearson, 2006.

[2] B. Cyganek, *Introduction to programming with C++ for engineers*. Chichester, West Sussex, UK ; Hoboken, NJ, USA: Wiley-IEEE Press, 2020.

[3] D. West, *Object thinking*. en Microsoft professional. Redmond, WA: Microsoft, 2004.

[4] *Lógica y programación orientada a objetos: un enfoque basado en problemas.* Medellín: Tecnologico de Antioquia, 2009.

[5] E. Balagurusamy, *Programación orientada a objetos con C++*, 4ª ed. Madrid: McGraw-Hill, 2007.

[6] M. Aniche, *Simple object-oriented design: create clean, maintainable applications*, [First edition]. Shelter Island, NY: Manning Publications, 2024.

[7] V. Bucheli, «AVATAR», Gtihub-Avatar. Accedido: 1 de enero de 2025. [En línea]. Disponible en: https://github.com/vbucheli/Avatar




[8] Y. Bugayenko, *Elegant objects. volume 1*, Version: 1.7. Palo Alto, California: Yegor Bugayenko, 9.
[9] R. C. Martin, J. Grenning, S. Brown, y K. Henney, *Clean Architecture: a craftsman's guide to software structure and design*. en Robert C. Martin series. Boston Columbus Indianapolis New York San Francisco Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo: Prentice Hall, 2018.